

8-BIT TIMER DESIGN

TABLE OF CONTENTS

1. Overview

a. Block diagram	3
b. Input/Outputs pin (LSI pin – CHIP PIN/PORT)	4
c. Register specification.....	5

2. Read/write register control

a. Input/Outputs pin.....	7
b. Functional/Protocol.....	7
c. The state machine operates through the following states.....	8
d. State machine control for APB protocol.....	9
e. Timing chart.....	9

3. Timer counter (Counter)

a. Design circuit.....	11
b. Functional/Protocol.....	11
c. Counter Logic functional.....	12

4. Clock Selection

a. Design circuit.....	13
b. Functional/Protocol.....	13
c. Internal Register.....	13

5. Timing Chart

a. Overflow Chart.....	14
b. Underflow chart.....	14
c. Count-up chart.....	14
d. Count_down chart.....	14

6. Verification Framework for 8-bit Timer

a. Overall Verification Architecture.....	16
b. Why Transaction-Based Verification Is Used.....	17
c. Mailbox-Based Communication.....	17
d. Driver: Converting Transactions Into APB Protocol.....	17
e. Monitor: Passive Bus Observation.....	18
f. Scoreboard and Reference Model.....	18
g. Functional Coverage.....	18
h. Base Test and Reusable Helpers.....	18
i. Testcase Phases.....	19
j. Regression and UCDB Coverage Merge.....	19
k. Summary.....	20

LIST OF FIGURES

Figure 1: Block diagram of TMR design (Timer)	3
Figure 2: Connection of 8-bit TMR.....	4
Figure 3: block diagram Apb controller.....	7
Figure 4: APB protocol state diagram.....	9
Figure 5: Read transfer with wait states.....	10
Figure 6: Write transfer with wait states.....	11
Figure 7: The Control logic diagram.....	11
Figure 8: The Clock Selection module logic diagram.....	13
Figure 9: Overflow timing.....	14
Figure 10: Underflow timing.....	14
Figure 11: Count-up timing.....	14
Figure 12: Count-down timing.....	14
Figure 13: The structure of a verification environment.....	16
Figure 14: The final coverage report of timer testbench	20
Figure 15: Testcases report.....	21

1. Overview

a. Block diagram

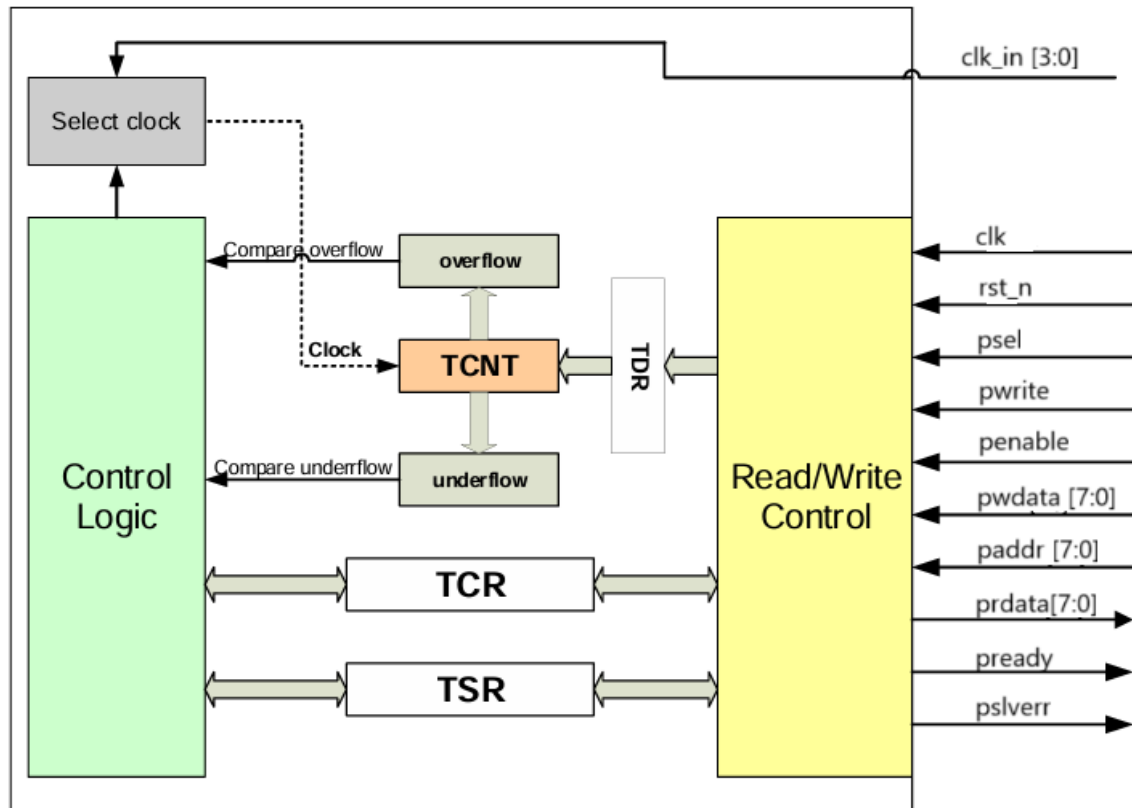


Figure 1: Block diagram of TMR design (Timer)

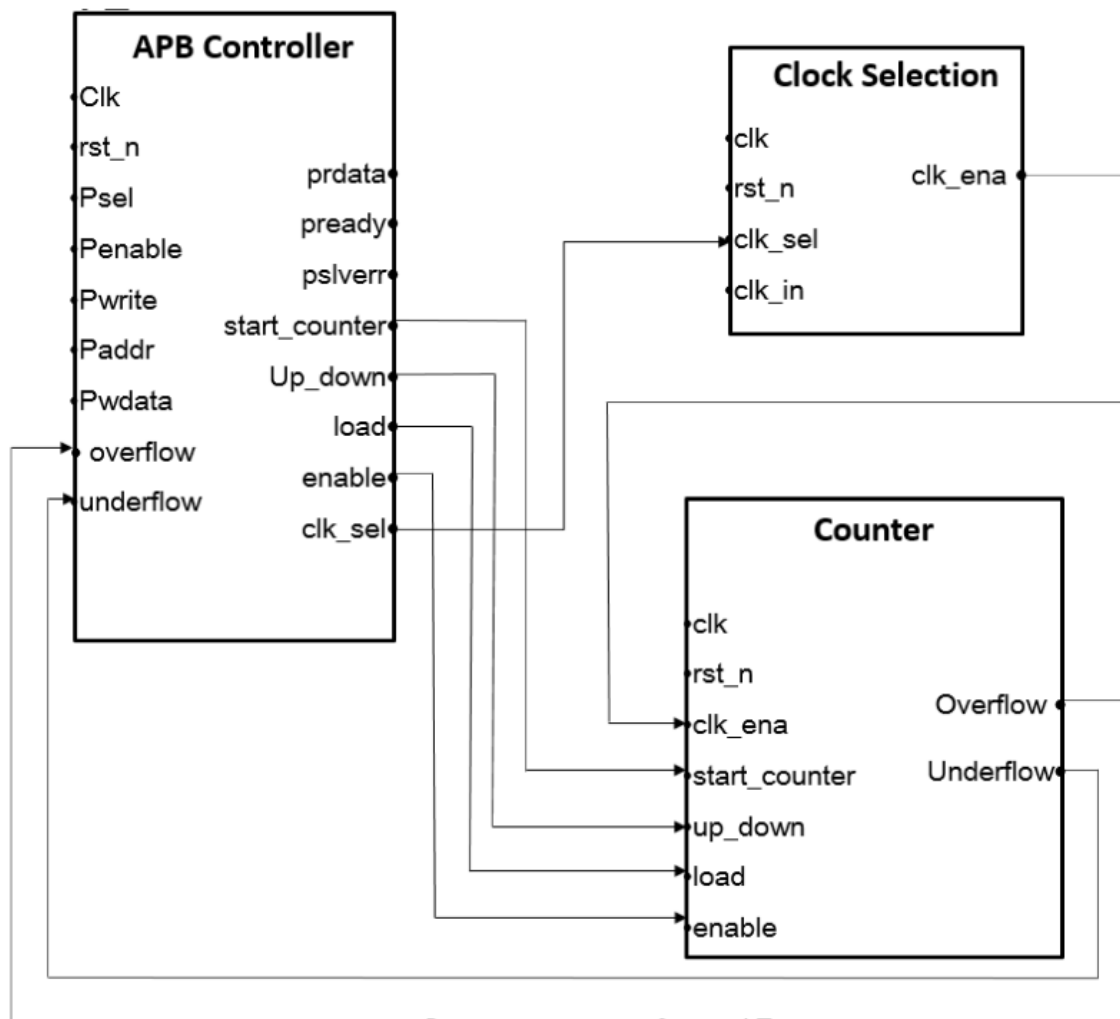


Figure 2: Connection of 8-bit TMR

b. Input/Outputs pin (LSI pin – CHIP PIN/PORT)

PORT name	Attribute	BIT WIDTH	Description
clk	Input	1	Is the system clock. All APB (Advanced Peripheral Bus) communication operations are synchronized to the rising edge of the clk signal.
rst_n	Input	1	Asynchronous and low-active reset signal. When rst_n = 0, all internal logic is returned to the default initialization state.
psel	Input	1	Chip Selection signal, high-active. When psel = 1, indicates that a transaction is in progress with the device (e.g. this TIMER block).
penable	Input	1	Access Phase control signal. This signal is activated after psel has been confirmed, and will be disabled when the transaction is completed.
pwrite	Input	1	Write control signal. When pwrite = 1, the system is performing a write operation to the internal register. if pwrite = 0, it means that a read operation is being performed.

paddr[7:0]	Input	8	APB address bus, is driven by the peripheral bus bridge unit.
pdata[7:0]	Input	8	Write data. This bus is driven by the peripheral bus bridge unit during write cycles when pwrite is HIGH.
clk_in[0]	Input	1	The counter clock has address 2'b00 . The internal selected clock block will select one of these signals as the master clock for the main counter.
clk_in[1]	Input	1	The counter clock has address 2'b01 . The internal selected clock block will select one of these signals as the master clock for the main counter.
clk_in[2]	Input	1	The counter clock has address 2'b10 . The internal selected clock block will select one of these signals as the master clock for the main counter.
clk_in[3]	Input	1	The counter clock has address 2'b11 . The internal selected clock block will select one of these signals as the master clock for the main counter.
pready	Output	1	Response signal from the slave (peripheral device). If pready = 0, the transaction will be delayed until the slave is ready to complete. when pready = 1, the transaction can be terminated normally.
pslverr	Output	1	Slave Error signal. When pslverr = 1, it means that an error occurred during data transmission on the APB bus.
prdata[7:0]	Output	8	Read data. This bus is driven by the peripheral bus bridge unit during write cycles when pwrite is LOW.
underflow	Output	1	Provided timer counter status when counter count from 8'h00 up to 8'hff
overflow	Output	1	Provided timer counter status when counter count from 8'hFF down to 8'h00

c. Register specification

• Timer data Register (TDR)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
tdr[7]	tdr[6]	tdr[5]	tdr[4]	tdr[3]	tdr[2]	tdr[1]	tdr[0]

Bit name	Attribute	Description
tdr[7:0]	R/W	Data is used to update the value of the counter.

• Timer counter control Register (TCR)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
tcr[7]	tcr[6]	tcr[5]	tcr[4]	tcr[3]	tcr[2]	tcr[1]	tcr[0]
Load	Reserved	Up/Down	Enable	Reserved	Reserved	Cks1	cks0

Bit name	Attribute	Description
Load[7]	R/W	Manual load data from TDR to TCNT when it is active High. 1: load data to TCNT 0: Normal operation.
[6]	Reserved	Reserved
Up/Down[5]	R/W	Control counter up or counter down 0: counter up

		1: counter down
Enable[4]	R/W	Initiate Timer counter control Register 0 : disable 1: enable
[3:2]	Reserved	Reserved
Clk_sel[1:0]	R/W	select internal clocks for circuit 2'b00: clk_in[0] 2'b01: clk_in[1] 2'b10: clk_in[2] 2'b11: clk_in[3]

• **Timer status Register (TSR):**

Bit name	R/W	Description
7:2	R	Reserved
S_TMR_UDF[1]	R/W	Provided timer counter status when counter count from 8'h00 up to 8'hff. This bit is only set by hardware, clear by software.
S_TMR_OVF[0]	R/W	Provided timer counter status when counter count from 8'hFF down to 8'h00. This bit is only set by hardware, clear by software.

2. Read/write register control

a. Input/Outputs pin

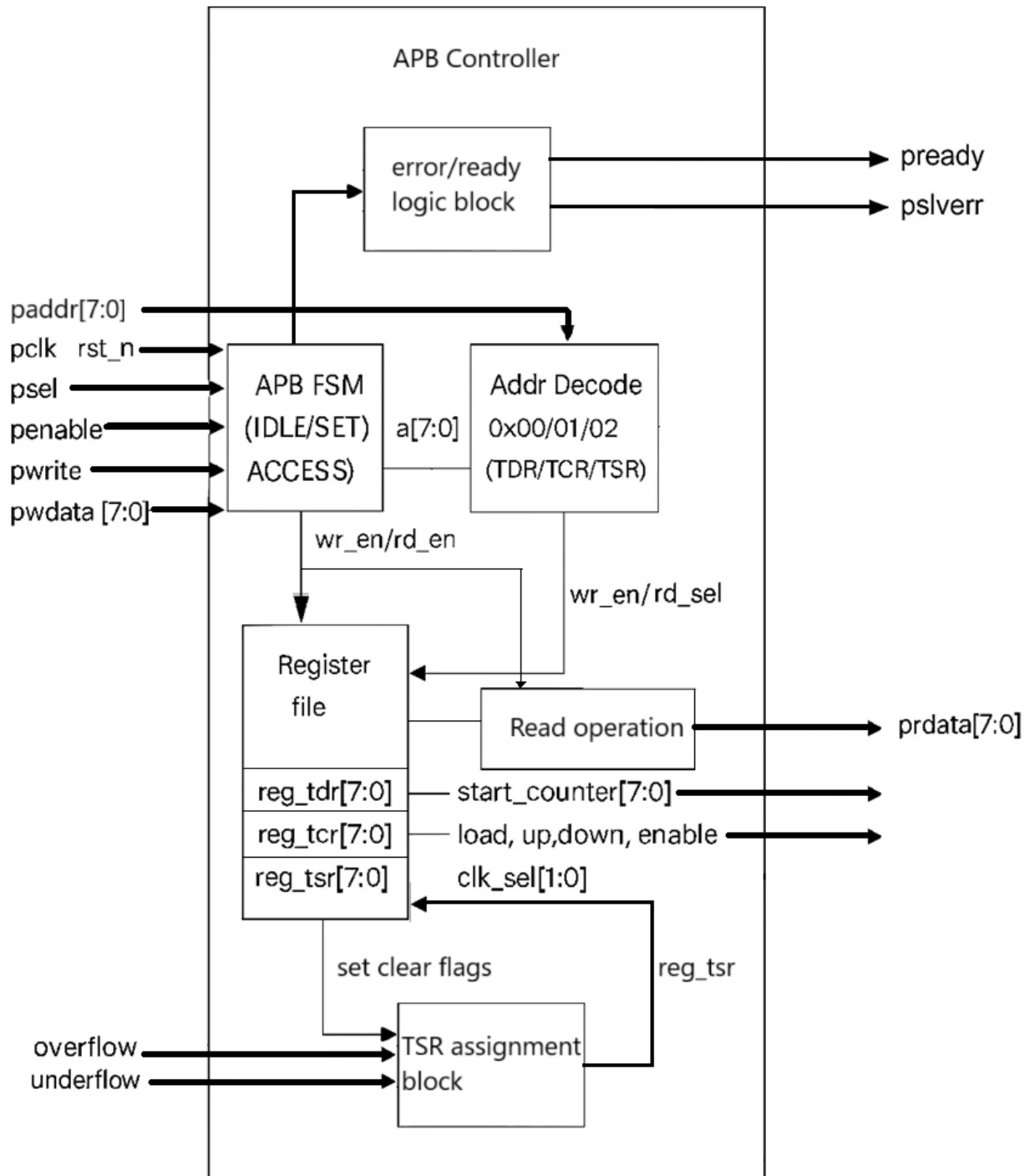


Figure 3: block diagram Apb controller

b. Functional/Protocol

The R/W Controller handles AMBA 3 APB protocol for internal register access.

- It decodes the register address (paddr) and distinguishes read/write operations using pwrite.
- On a write, it transfers pwrite to the addressed register.
- On a read, it places the register data onto prdata.
- it asserts pready to signal operation completion and pslverr if an invalid address is accessed.
- all operations are synchronized with clk and reset by rst_n

The Apb controller module uses the APB protocol to perform Read/Write value on three registers described below:

- TDR register: perform full Read/Write transaction.
- TCR register: perform full Read/Write transaction.
- TSR register: perform Read transaction on normal operation, the Write transactions are only allowed when Overflow, Underflow, or both statuses exist in the TSR register.

Read/write register APB protocol spec in here:

PORT name	Attribute	BIT WIDTH	Description
clk	Input	1	clk signal.
rst_n	Input	1	Asynchronous and low-active reset signal.
psel	Input	1	Chip Selection signal, high-active. When psel = 1.
penable	Input	1	Access Phase control signal.
pwwrite	Input	1	Write control signal.
paddr[7:0]	Input	8	APB address bus.
pwrite[7:0]	Input	8	Write data.
clk_in[0]	Input	1	The counter clock has address 2'b00 .
clk_in[1]	Input	1	The counter clock has address 2'b01 ..
clk_in[2]	Input	1	The counter clock has address 2'b10 ..
clk_in[3]	Input	1	The counter clock has address 2'b11 ..
pready	Output	1	Response signal from the slave (peripheral device).
pslverr	Output	1	Slave Error signal.
prdata[7:0]	Output	8	Read data from peripheral.
start_counter	Output	8	Value of the TDR register
load	Output	1	Load signal from bit 7 of TCR register.
up_down	Output	1	Control signal for up/down from bit 5 of TCR register.
enable	Output	1	Enable signal from bit 4 of TCR register.
clk_sel [1:0]	Output	2	Clock select signal from bits [1:0] of TCR register.

c. The state machine operates through the following states

- IDLE: This is the default state of the APB.
- SETUP: When a transfer is required the bus moves into the SETUP state, where the appropriate select signal, PSEL, is asserted. The bus only remains in the SETUP state for one clock cycle and always moves to the ACCESS state on the next rising edge of the clock.

- **ACCESS:** The enable signal, PENABLE, is asserted in the ACCESS state. The address, write, select, and write data signals must remain stable during the transition from the SETUP to ACCESS state.

Exit from the ACCESS state is controlled by the PREADY signal from the slave:

- If PREADY is held LOW by the slave then the peripheral bus remains in the ACCESS state.
- If PREADY is driven HIGH by the slave then the ACCESS state is exited and the bus returns to the IDLE state if no more transfers are required. Alternatively, the bus moves directly to the SETUP state if another transfer follows.

d. State machine control for APB protocol

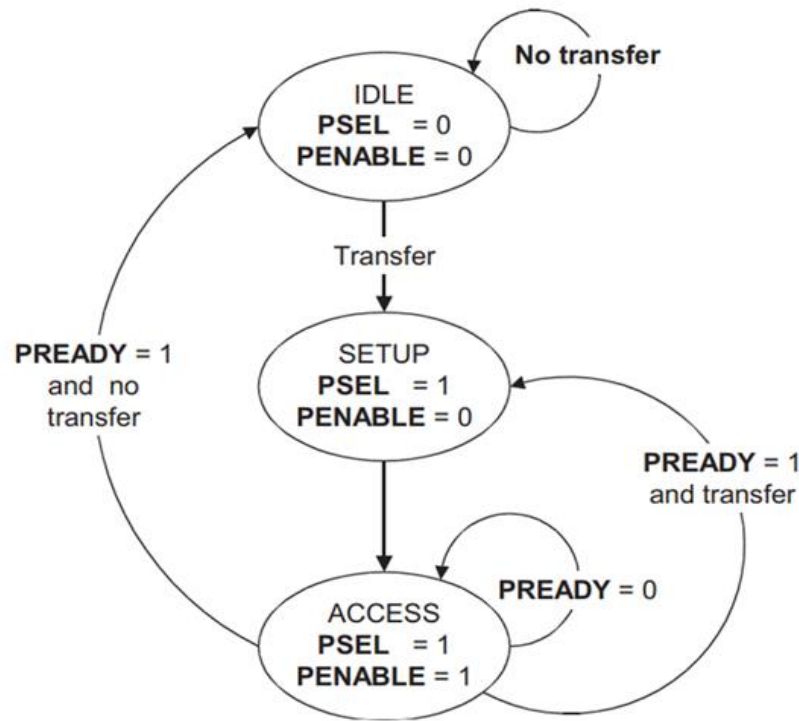


Figure 4: APB protocol state diagram

e. Timing chart

Read transaction with wait state: Referring to the following diagram, read transactions to the APB interface follow these steps:

- At T1, a read transfer starts with address ADDR1, PWRITE asserted LOW and select signal PSEL registered at the rising edge of the core clock. This is the setup phase of the write transfer.
- At T2, PENABLE is registered at the rising edge of the core clock and held HIGH until the HBM2 controller asserts PREADY HIGH. The values of PADDR, PSEL, PENABLE and PWRITE must remain unchanged while PREADY remains LOW.

- At Tx, when PREADY goes HIGH, Read Data is available on the PRDATA bus. PREADY stays HIGH only for one clock cycle.
- PENABLE is deasserted at the end of the transfer. The select signal PSEL, is also deasserted unless the transfer is to be followed immediately by another transfer.

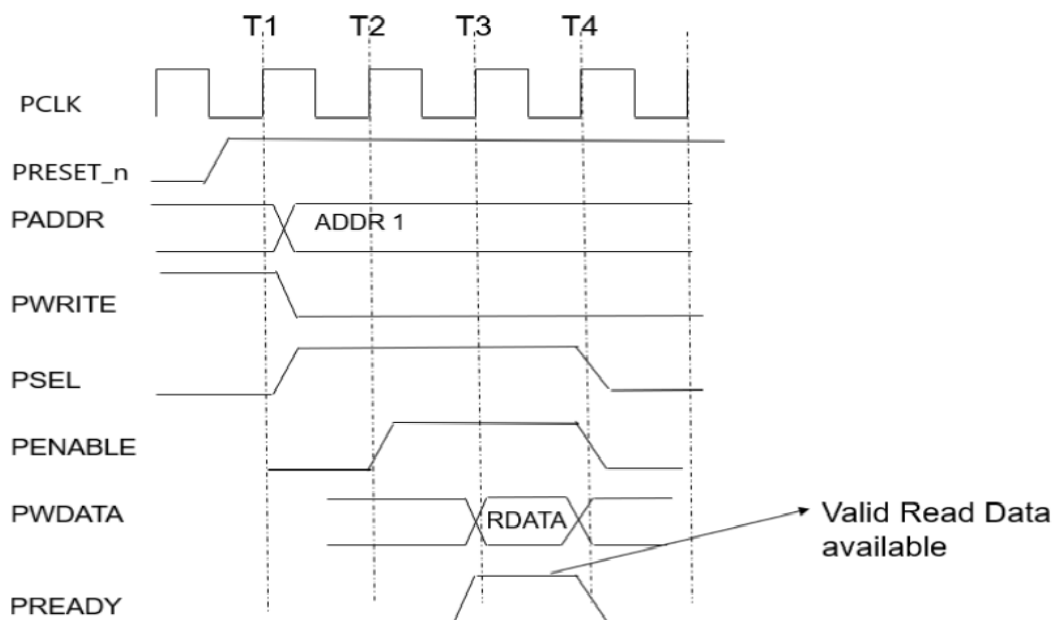


Figure 5: Read transfer with wait states

Write transaction with wait state: Referring to the following diagram, write transactions to the APB interface follow these steps:

- At T1, a write transfer starts with address ADDR1, write data DATA1, write signal PWRITE and select signal PSEL registered at the rising edge of the core clock. This is the setup phase of the write transfer.
- At T2, PENABLE is registered at the rising edge of the core clock and held HIGH until the HBM2 controller asserts PREADY HIGH. The values of PADDR, PSEL, PENABLE, PWDATA, PSTRB and PWRITE must remain unchanged while PREADY remains LOW.
- At Tx, when PREADY goes HIGH, the write transaction completes at the next rising edge of the core clock. This indicates the end of the Write Access Phase. PREADY stays HIGH only for one clock cycle.
- PENABLE is deasserted at the end of the transfer. The select signal PSEL, is also deasserted unless the transfer is to be followed immediately by another transfer.

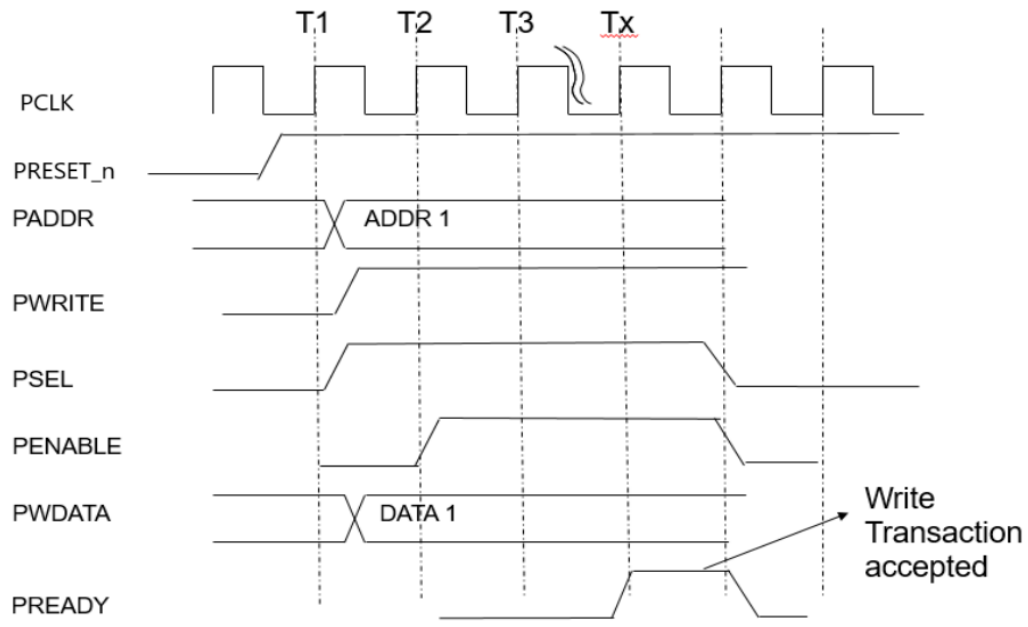


Figure 6: Write transfer with wait states

3. Timer counter (Counter)

a. Design circuit

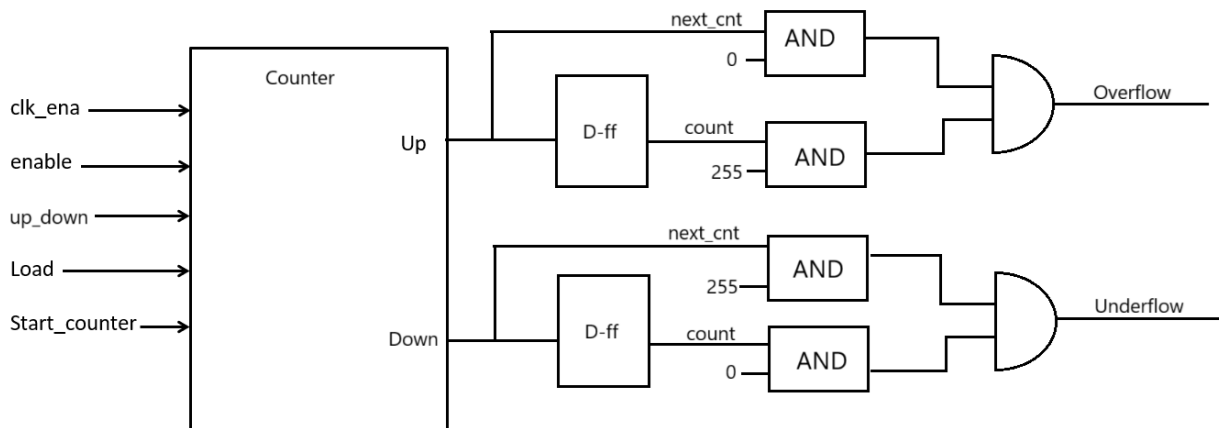


Figure 7: The Control logic diagram

b. Functional/Protocol

Input signal:

Signal	Type	Description
--------	------	-------------

clk_ena	Clock	System clock used to synchronize signals (especially inside select_clock block).
Load	Control bit	Load bit (in TCR): 1 → use value from TDR, 0 → use default value (8'h00 or 8'hFF depending on count direction).
Start_counter[7:0]	Data	Value written by CPU into TDR register, used as the start value if LOAD=1.
up_down	Control bit	Bit to select count direction: 0 → count up, 1 → count down.
enable	Control bit	Enable bit: 1 → enable counting, 0 → stop counting.

Output signals:

Signal	Type	Description
Overflow	Status flag	Overflow status bit (OVF) sent to TSR register.
Underflow	Status flag	Underflow status bit (UDF) sent to TSR register.

c. Counter Logic functional:

When load is active, count is loaded with the value from start_counter.

When enable and clk_ena are active:

- If up_down is 1, count is incremented by 1.
- If up_down is 0, count is decremented by 1.

Update Previous Counter Value: The value of next_cnt is assigned from count.

Overflow & Underflow Logic:

- overflow: Activated when next_cnt is 0 and count is 255.
- underflow: Activated when next_cnt is 255 and count is 0.

4. Clock Selection

a. Design circuit

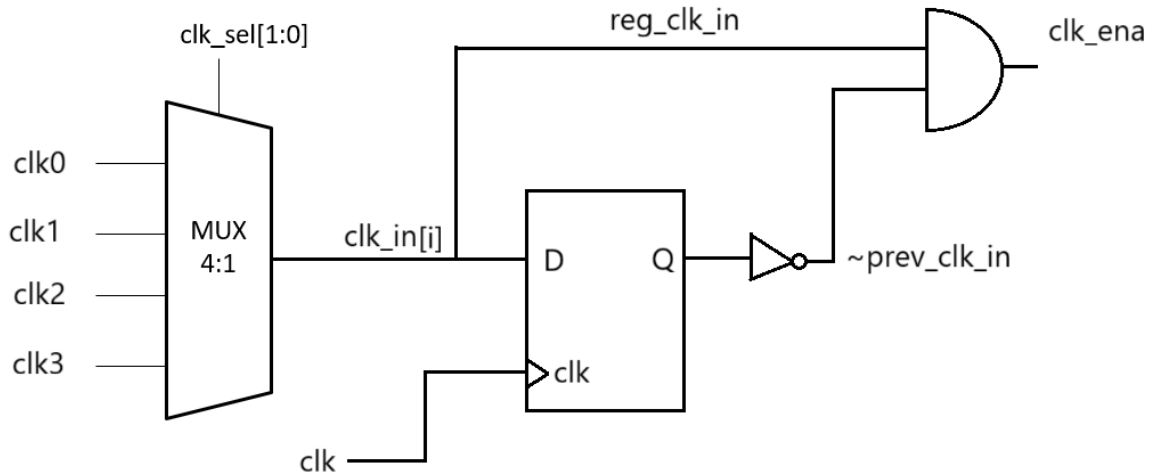


Figure 8: The Clock Selection module logic diagram

b. Functional/Protocol

Input signal:

Signal	Type	Description
clk_sel[1:0]	Data	2-bit selection signal used to choose which clock from clk_in
clk_in[3:0]	Clock	4-bit input representing four different clock signals

Output signals:

Signal	Type	Description
clk_ena	Status flag	The enabled clock signal, determined by the clk_sel value

c. Internal Register

clk_in_prev [3:0]:

- This is a 4-bit register that stores the previous value of clk_in.
- It is delayed by one clock cycle.

clk_in_prev <= clk_in;

clk_in_ena [3:0]:

- This is a 4-bit register that stores the clock-enable signals for each of the clocks in clk_in.
- The signals are generated by detecting the rising edges of the individual clocks in clk_in.

clk_in_ena <= clk_in & !clk_in_prev;

5. Timing Chart

a. Overflow Chart

At the moment Count is 8'hFF, the next $\uparrow$ edge causes Count to flip back to 8'h00. Overflow is set immediately this cycle and remains at 1 (until software/LOAD/reset clears).

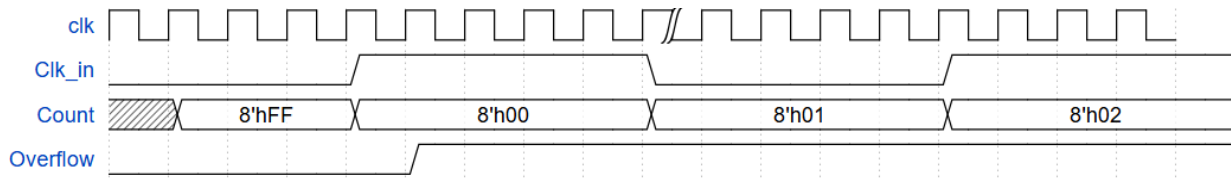


Figure 9: Overflow timing

b. Underflow chart

At the moment Count is 8'h00, the next $\uparrow$ edge causes Count to flip to 8'hFF. Underflow is set at this wrap cycle and remains at 1 (until cleared).

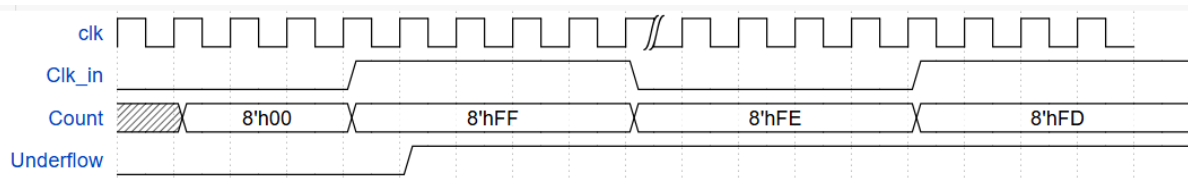


Figure 10: Underflow timing

c. Count-up chart

- Up/Dw=1 selects up counting mode.
- Clk_in opens the window for counting; each $\uparrow$ edge of clk inside the window increases Count by 1. Illustration: CNT-1 $\rightarrow$ CNT $\rightarrow$ CNT+1 $\rightarrow$ CNT+2 through consecutive Clk_in windows.

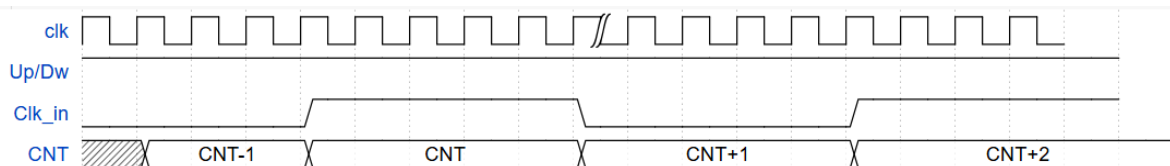


Figure 11: Count-up timing

d. Count_down chart

- Up/Dw=0 selects down counting mode.
- In the Clk_in=1 region, each edge $\uparrow$ of clk decreases Count by 1. Illustration: CNT+1 $\rightarrow$ CNT $\rightarrow$ CNT-1 $\rightarrow$ CNT-2 through the counting windows.

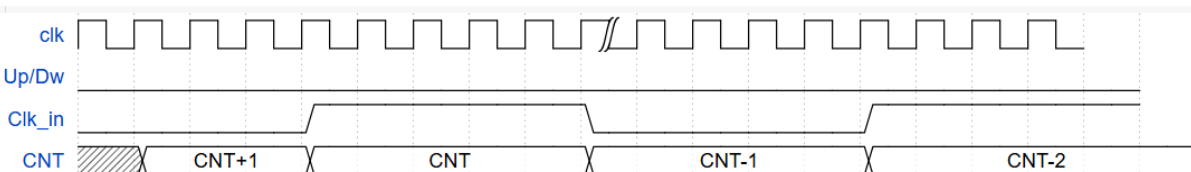


Figure 12: Count-down timing

6. Verification Framework for 8-bit Timer

This chapter documents the non-UVM verification framework used to verify the 8-bit timer design. The goal is to explain not only what each verification component does, but also how the components cooperate to create, drive, observe, check, and measure APB timer behavior in an industry-style flow.

Verification goal: The testbench must prove that register access, APB error response, count-up/count-down behavior, pause/resume, reset/load behavior, fake overflow/underflow prevention, and functional coverage goals are correctly handled by the timer RTL.

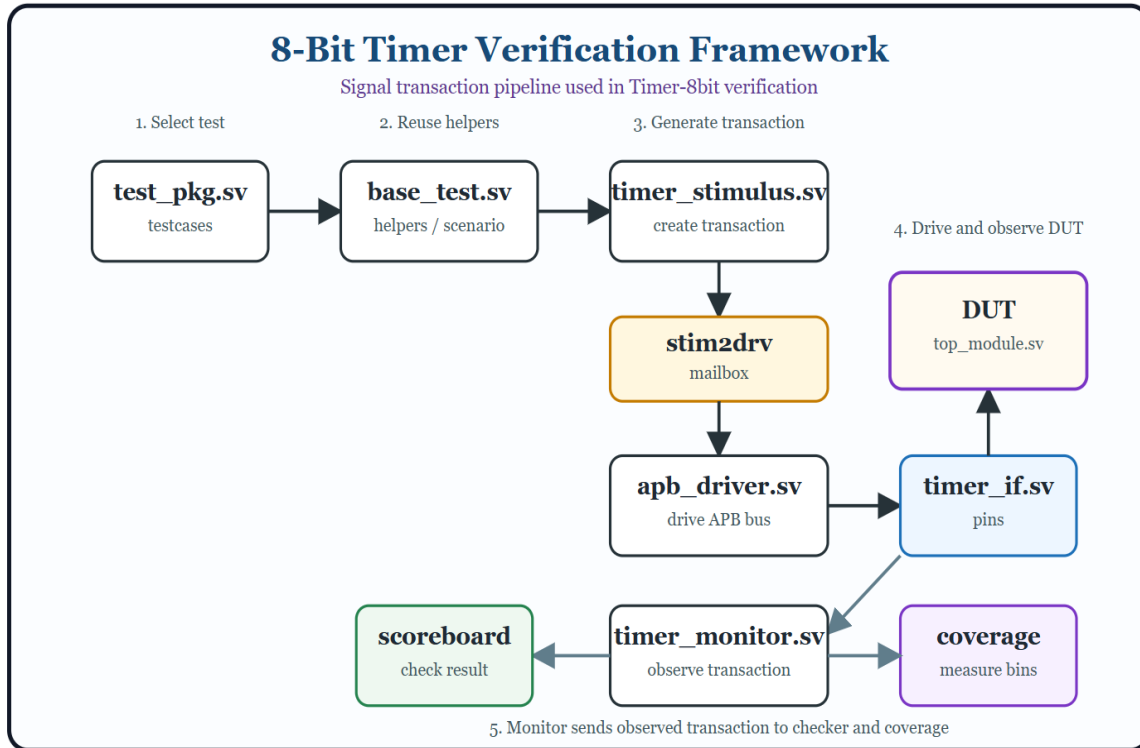


Figure 13: The basic structure of a verification environment

a. Overall Verification Architecture

The project uses a layered non-UVM architecture. Although it does not use the UVM library, it follows the same core industrial concepts: transaction-level stimulus, driver-based pin control, passive monitoring, scoreboard checking, functional coverage collection, reusable base tests, and regression automation.

Layer	Main files/classes	Responsibility
Top	tb_top.sv	Instantiate DUT/interface, generate clk/rst/clk_in, select testcase by +TESTNAME.
Interface	timer_if.sv	Bundle APB signals, timer clocks, reset, and TB modport/DUT modport.
Transaction	apb_transaction.sv	Represent one abstract APB read/write operation.
Stimulus	timer_stimulus.sv	Create transaction objects and send them through mailbox.
Driver	apb_driver.sv	Convert transaction into cycle-accurate APB pin activity.
Monitor	timer_monitor.sv	Observe APB bus and reconstruct completed transactions.

Scoreboard	timer_scoreboard.sv	Maintain expected model and compare DUT response.
Coverage	timer_coverage.sv	Sample functional coverage from observed transactions.
Environment	timer_environment.sv	Build and connect driver, monitor, scoreboard, coverage, and mailboxes.
Test	base_test.sv and testcase classes	Provide reusable helpers and implement directed scenarios.

The transaction path can be understood as:

testcase/base_test -> stimulus -> mailbox -> driver -> interface pins -> DUT

The checking path is passive and independent:

DUT/interface pins -> monitor -> mailbox -> scoreboard and coverage

b. Why Transaction-Based Verification Is Used

A transaction is a high-level object that describes the intent of an APB access, for example: write address 0x01 with data 0x30, or read address 0x02. The testcase does not directly toggle psel, penable, paddr, pwrite, or pwrdata. Instead, it creates transactions. This separation makes tests easier to read, reduces duplicated protocol code, and allows the APB protocol implementation to be centralized in the driver.

apb_transaction field	Meaning
write	1 for APB write, 0 for APB read.
addr[7:0]	APB register address: TDR=0x00, TCR=0x01, TSR=0x02, invalid address >0x02.
data[7:0]	Write data for write transaction. For read transaction this is usually 0.
rsp_data[7:0]	Read response captured by monitor/driver path.
rsp_err	PSLVERR response. Expected to be 1 for invalid address.
sample_time	Simulation timestamp used for debug/logging.
name	Human-readable transaction name used in logs.

c. Mailbox-Based Communication

The framework uses SystemVerilog mailboxes to pass transaction handles between components. A mailbox decouples producer and consumer timing. The stimulus can generate an APB transaction at transaction level, and the driver can retrieve it later when it is ready to drive the bus.

- stim2drv mailbox: carries transactions from timer_stimulus to apb_driver.
- mon2sb mailbox: carries monitored transactions from timer_monitor to timer_scoreboard.
- mon2cov mailbox: carries monitored transactions from timer_monitor to timer_coverage.

d. Driver: Converting Transactions Into APB Protocol

The APB driver is the active component. It consumes apb_transaction objects from stim2drv and performs the APB setup/access sequence on timer_if. For a write, it drives psel, pwrite, paddr, pwrdata, then asserts penable in the access phase. For a read, it drives psel, pwrite=0 and paddr, then samples prdata after the DUT indicates the transfer is complete.

Important timing lesson: During debug, an extra sampling cycle after pready was required because the RTL response data/error became stable one cycle later than a textbook zero-wait APB response. The driver and monitor were aligned to the actual RTL timing to avoid false failures.

e. Monitor: Passive Bus Observation

The monitor does not drive any signal. It watches the APB interface and creates a transaction whenever it detects a completed APB transfer. This is important because the scoreboard should check what the DUT actually saw on the bus, not what the stimulus intended to send.

- It captures write/read direction from pwrite.
- b. It captures address and write data from paddr/pwdata.
- It captures read response data from prdata.
- It captures error response from pslverr.
- It broadcasts the reconstructed transaction to scoreboard and coverage.

f. Scoreboard and Reference Model

The scoreboard is the main checker. It implements a simple reference model of the timer registers and status behavior. For APB accesses, it predicts PSLVERR and PRDATA. For timer behavior, it models expected counter progress based on TCR fields and selected clock edges.

Checked behavior	Expected model rule
Valid addresses	TDR=0x00, TCR=0x01, TSR=0x02 should not assert PSLVERR.
Invalid addresses	Any address greater than 0x02 should assert PSLVERR and return 0 on read.
TDR read/write	Readback should match the last valid TDR write.
TCR read/write	Readback should match writable TCR fields according to RTL behavior.
TSR clear	TSR status bits are sticky and are cleared by writing 1 to the corresponding bit.
Overflow	Count-up from 0xFF to 0x00 sets TSR[0].
Underflow	Count-down from 0x00 to 0xFF sets TSR[1].
Reset	Reset clears TDR, TCR, TSR, expected count, and selected-clock history.

g. Functional Coverage

Functional coverage measures whether important verification scenarios have been exercised. Unlike scoreboard checking, coverage does not decide pass/fail. It answers the question: have we stimulated all meaningful cases?

Coverage item	Purpose
APB read/write	Ensure both read and write transfers are exercised.
Register address bins	Ensure TDR, TCR, TSR, and invalid address accesses are covered.
PSLVERR bins	Ensure both normal access and error response are observed.
Data bins	Ensure low/mid/high/random data values are written/read.
Crosses	Ensure combinations such as write-to-register, read-from-register, and invalid access are hit.

h. Base Test and Reusable Helpers

base_test.sv is the reusable test foundation. It builds the environment, initializes APB signals, waits for reset release, runs testcase-specific main_run(), waits for the environment to become idle, and prints the final report.

- write_reg/read_reg hide APB transaction generation from testcase code.
- load_counter writes TDR, asserts TCR load, then writes the next TCR operating value.
- wait_selected_clk_edges counts selected clk_in rising-edge events using clk_sel.

- expect_status_bit_clear and expect_status_bit_set check TSR status intent.
- pause helpers verify EN=0 prevents counting during pause.
- reset helpers verify mid-test reset returns registers to default and allows reconfiguration.
- fake-event helpers verify disabled load/write operations do not create false OVF/UDF events.

i. Testcase Phases

Phase	Testcases	Verification intent
Phase 1 - Register	1-5	Read/write TDR/TCR/TSR, invalid address, mixed valid/invalid access.
Phase 2 - Function	6-13	Count-up/count-down across pclk2/pclk4/pclk8/pclk16 using fork-join timing checks.
Phase 3 - Pause	14-15	Disable EN during counting, prove no OVF/UDF during pause, then resume and complete event.
Phase 4 - Reset/Load	16-19	Reset during operation, check default registers, reload old/new values, verify post-reset operation.
Phase 5 - Fake Event	20-21	Load boundary values while EN=0 and verify no fake overflow/underflow is set.

10. Important Debug Lessons From This Project

- Sticky status bits must be cleared before starting a new scenario; otherwise the next test may read an old OVF/UDF flag.
- For timing-sensitive checks, avoid checking a scoreboard mirror after long wait_idle delays because the timer may continue running.
- The APB monitor and driver must sample according to the actual RTL response timing, not only the ideal APB timing diagram.
- A reset testcase must reset the scoreboard mirror state as well as the DUT, otherwise a correct DUT can appear to fail.
- Fake overflow/underflow tests are valuable because they prove that status flags are generated by real counter wrap events, not by loading boundary data.

j. Regression and UCDB Coverage Merge

For project-level coverage, each testcase is run with coverage enabled and saved into an individual UCDB database. After all 21 tests complete, vcover merge combines all UCDB files into one database, all_tests.ucdb. This gives a single coverage view for the entire regression.

- Compile with coverage enabled: `vlog.exe -coveropt 3 +cover=bcesft +acc -f compile.f`
- Run each test with `vsim.exe -coverage` and save one UCDB per testcase.
- Create `ucdb_files.txt` listing all testcase UCDB files.
- Merge with: `vcover.exe merge -inputs cov/ucdb_files.txt cov/all_tests.ucdb`
- Generate text or HTML reports from `cov/all_tests.ucdb`.

All the above commands have been combined and condensed into a single command so that users can run the entire process of printing the coverage report, which is saved in the Makefile. By entering the following into the Terminal interface: `make cov_regress`

k. Summary

Questa Coverage Report

Number of tests run:	21
Passed:	20
Warning:	0
Error:	1
Fatal:	0

[List of tests included in report...](#)

[List of global attributes included in report...](#)

Coverage Summary by Structure:

Design Scope	Coverage
tb_top	93.44%
tif	100.00%
dut	93.52%
timer_pkg	94.21%
apb_transaction	100.00%
apb_driver	100.00%
timer_monitor	100.00%
timer_stimulus	44.00%
timer_scoreboard	94.57%
timer_coverage	96.87%
timer_environment	100.00%
test_pkg	85.60%
base_test	81.41%

Coverage Summary by Type:

Total Coverage:					92.78%	92.13%
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Covergroups	58	51	7	1	87.93%	93.75%
Statements	767	727	40	1	94.78%	94.78%
Branches	154	125	29	1	81.16%	81.16%
FEC Expressions	20	20	0	1	100.00%	100.00%
FEC Conditions	23	21	2	1	91.30%	91.30%
Toggles	398	374	24	1	93.96%	93.96%
FSMs	8	7	1	1	87.50%	90.00%
States	3	3	0	1	100.00%	100.00%
Transitions	5	4	1	1	80.00%	80.00%

test_pkg	85.60%
base_test	81.41%
tdr_test	70.83%
tcr_test	87.50%
tsr_test	87.50%
null_address_test	71.87%
mixed_address_test	92.58%
countup_pclk2_test	100.00%
countup_pclk4_test	100.00%
countup_pclk8_test	100.00%
countup_pclk16_test	100.00%
countdw_pclk2_test	100.00%
countdw_pclk4_test	100.00%
countdw_pclk8_test	100.00%
countdw_pclk16_test	100.00%
countup_pause_countup_test	100.00%
countdw_pause_countdw_test	100.00%
countup_reset_countdw_pclk2_test	100.00%
countdw_reset_countup_pclk2_test	100.00%
countup_reset_load_countdw_pclk2_test	100.00%
countdw_reset_load_countdw_pclk2_test	100.00%
fake_underflow_test	100.00%
fake_overflow_test	100.00%

Report generated by [Questa](#) on Tuesday 12 May 2026 18:46:10

Figure 14: The final coverage report of timer testbench

The test results showed an overall coverage rate of 92.13%. All test cases passed the tests and achieved high coverage. However, one test failed in the `null_address_test` because the test required generating invalid addresses, and the system detected this error appropriately. Therefore, this case is considered a passed, like the other test cases.

Tests included in report:

Testname	User	CPU Time	SIM Time	Status
countdw_pause_countdw_test	pc	00:00:00.00	22760.00 ps	OK
countdw_pclk2_test	pc	00:00:00.00	19300.00 ps	OK
countdw_pclk4_test	pc	00:00:00.02	32220.00 ps	OK
countdw_pclk8_test	pc	00:00:00.02	58100.00 ps	OK
countdw_pclk16_test	pc	00:00:00.02	109460.00 ps	OK
countdw_reset_countup_pclk2_test	pc	00:00:00.00	31620.00 ps	OK
countdw_reset_load_countdw_pclk2_test	pc	00:00:00.02	31100.00 ps	OK
countup_pause_countup_test	pc	00:00:00.02	23260.00 ps	OK
countup_pclk2_test	pc	00:00:00.02	19020.00 ps	OK
countup_pclk4_test	pc	00:00:00.00	31660.00 ps	OK
countup_pclk8_test	pc	00:00:00.02	56980.00 ps	OK
countup_pclk16_test	pc	00:00:00.02	107220.00 ps	OK
countup_reset_countdw_pclk2_test	pc	00:00:00.00	28100.00 ps	OK
countup_reset_load_countdw_pclk2_test	pc	00:00:00.02	30240.00 ps	OK
fake_overflow_test	pc	00:00:00.00	2680.00 ps	OK
fake_underflow_test	pc	00:00:00.00	2680.00 ps	OK
mixed_address_test	pc	00:00:00.00	6170.00 ps	OK
null_address_test	pc	00:00:00.00	5570.00 ps	ERROR
tcr_test	pc	00:00:00.00	6170.00 ps	OK
tdr_test	pc	00:00:00.00	6170.00 ps	OK
tsr_test	pc	00:00:00.02	6170.00 ps	OK

Figure 15: testcases report

This non-UVM verification framework gives the project an industry-style structure while staying simple enough for a first complete verification project. The key idea is separation of concerns: tests describe what scenario to run, stimulus creates transactions, driver controls pins, monitor observes the DUT, scoreboard checks correctness, coverage measures completeness, and regression merges results across all testcase phases.